



Demo 3



Project: Graph Editor

Student: Ha Thanh Chu



Sprint 3 - backlog

Goal: The user will be able to generate reports and export file

ID	User story/Backlog Item	How to Demo?
US6	As a user, I want to compute centrality metrics for nodes, so that I can identify the most important nodes in the graph	Page rank, degree
US7	As a user, I want to detect communities in the graph, so that I can see clusters of related nodes	Community detection, connected components
US8	As a user, I want to export the graph, so that I can use it in reports, presentations, or other tools.	Export file to JSON, PNG, GraphML

Demo 3 - Something new?

- Canvas and UI perfectly aligned, no mismatched screens
- Search nodes by **ID or label**; triggered by magnifying glass click
- Node content: **text lists**
- Edges and parentID sorted by label
- **Unlimited undo/redo**



Undo

Redo

Add Node

Select action type

Add node + edge

Node

Node ID

Enter Node ID

Label

Enter node label

Parent ID (if not root - mandatory with hierarchy layout)

No parent

Additional properties

Enter additional properties, separate by ; e.g., age:30;city:NY

Color: 

Edge

Source Node

✓ Select Source

Alice (id: 1)

Anaya (id: 14)

Bob (id: 2)

Carol (id: 3)

Dave (id: 4)

Eve (id: 5)

Frank (id: 6)

Grace (id: 7)

Heidi (id: 8)

Ivan (id: 9)

Leo (id: 13)

Lova (id: 15)

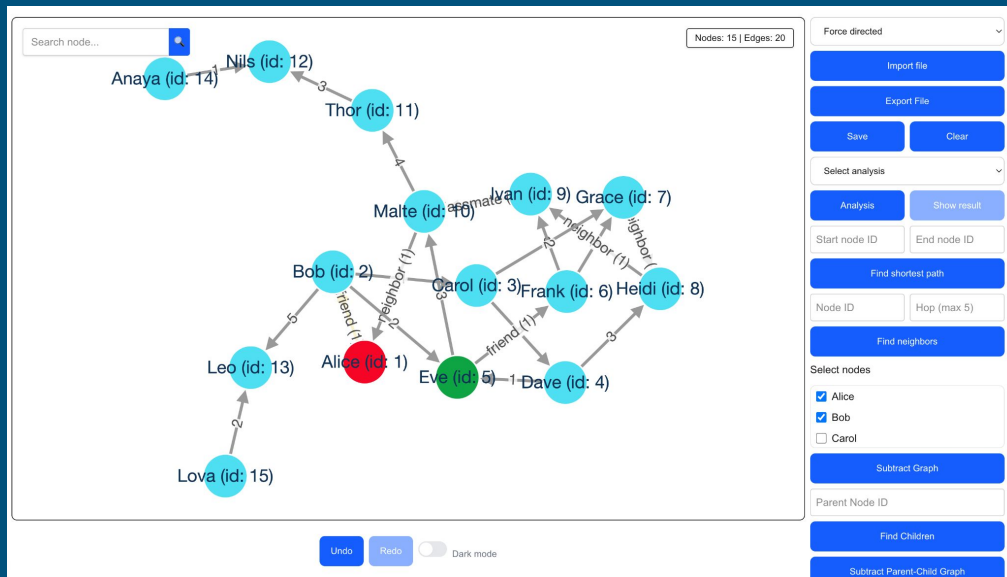
Malte (id: 10)

Nils (id: 12)

Thor (id: 11)

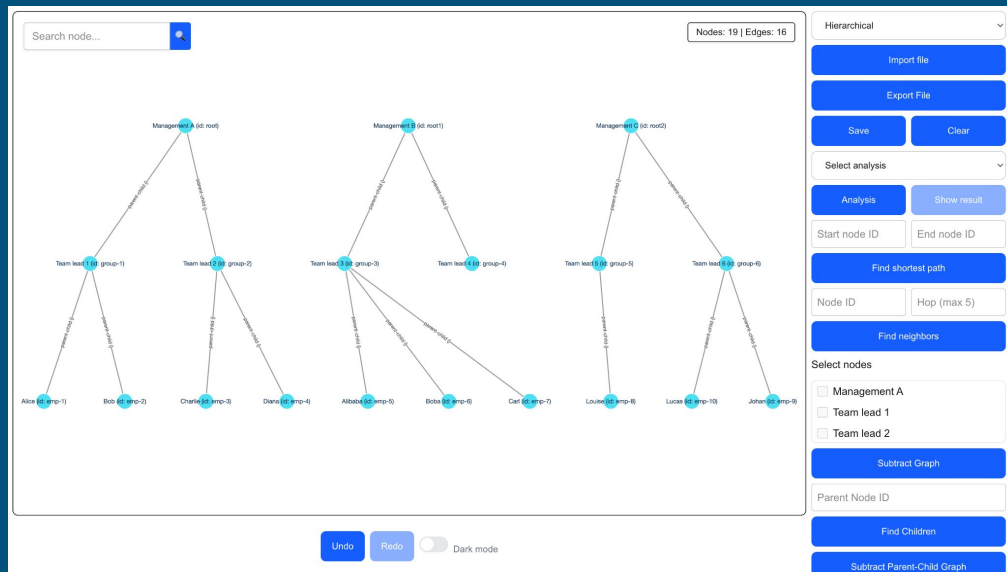
Demo 3 - Something new?

- Empty graph: only **add node** or **import file**, other features hidden
- **Auto-save** and save success/failure toast notifications
- Export graph: **JSON, PNG, GraphML**
- **Clear graph** button (database + canvas)
- Graph analysis: **degree centrality, PageRank, community detection** with node size & color highlight



Demo 3 - Something new?

- Hierarchical graph: **disable node selection for subgraph**, filter by parentID, generate parent-child subgraph
- Hierarchy layout: **edit/add edge disabled**
- Most functions use **unique Node ID** to avoid duplicate label conflicts
- **Toast notifications** instead of alerts



Customer acceptance tests

ID	Requirement	Test Scenario
1	Graph creation and editing	Create a new graph manually. Add, edit, and delete nodes and edges. Undo and redo edits.
2	Import existing graph data	Import graph data from JSON files.
3	Content format options	Users can input box content details in a label as text lists or audio note(s) or image(s)
4	Export graph	Export graph to GraphML and PNG format.

Customer acceptance tests

ID	Requirement	Test Scenario
5	Node and edge attribute editing	Modify node labels and edge weights through property panel or similar appropriate.
6	Graph layout options	Apply at least two layout algorithms (e.g., none-force-directed and force-directed).
7	Zooming and panning	Use mouse or UI controls to zoom in/out and pan across large graphs.
8	Search and filtering	Perform text-based node search and attribute-based filter.

Customer acceptance tests

ID	Requirement	Test Scenario
9	Shortest-path finder	Select two nodes and compute shortest path
10	Analytics: Degree and PageRank	Run degree centrality and PageRank analysis.
11	Manual node positioning	Drag nodes manually and snap to grid.
12	Save and load session	Save current graph state and reopen later.

Customer acceptance tests

ID	Requirement	Test Scenario
13	Performance test on large graph	Load graph with 5k+ nodes and 10k+ edges. => only 1000
15	Database connection Connect to a database	Neo4J
17	User interface design	The editor uses a visually appealing layout without browser alerts (toast)

Thank you for listening!